

TEORÍA DE CONJUNTOS

Manual Completo de Exposición y Banco de Preguntas

INFORMACIÓN DEL PROYECTO:

- Tema: Calculadora de Conjuntos y Diagramas de Venn Interactivos
- Tecnologías: Vue 3 (Composition API), TypeScript (Strict), Tailwind CSS v4, Vitest
- Integrantes: Jordy (Sublíder), Junior, Mike, Sergio, Fernando, Alejandro

1. ESTRUCTURA Y FLUJO GENERAL DEL PROYECTO

El proyecto se divide estrictamente en dos capas arquitectónicas:

1. Capa Lógica (El Motor): Programado en TypeScript en 'src/lib/sets/operations.ts'. No tiene nada visual, es puramente matemático. Esto permite probar el código de forma automática sin cargar la web.
2. Capa Visual (La Interfaz): Programada en Vue 3 en 'src/pages/conjuntos/index.vue'. Captura los datos del usuario, llama al motor para calcular, e ilumina un diagrama SVG dinámicamente según la operación.

Control de Versiones (Git):

Trabajamos bajo flujo de trabajo profesional por ramas. Cada Sprint tuvo su propia rama:

- equipo-4-linus-propuesta (Sprint 1)
- equipo-4-linus-motor (Sprint 2)
- equipo-4-linus-ui (Sprint 3)
- equipo-4-linus-deploy (Sprint 4)

2. EXPLICACIÓN DEL CÓDIGO (EL MOTOR MATEMÁTICO)

Uso de Generics <T> y Sets de ES6:

En lugar de usar arreglos comunes, usamos el tipo Set nativo de JavaScript porque asegura que los elementos sean únicos (no permite duplicados) y permite búsquedas rápidas mediante .has().

Usamos Generics <T> en las funciones para permitir que el motor funcione con cualquier tipo de datos (conjuntos de números, letras, palabras o incluso objetos), garantizando robustez de código.

Explicación de las Funciones Clave:

- Unión ($A \cup B$): Junta todos los elementos usando desestructuración: `new Set([...a, ...b])`.
- Intersección ($A \cap B$): Filtra A dejando solo los que existen en B: `new Set([...a].filter(x => b.has(x)))`.
- Diferencia ($A - B$): Filtra A dejando solo los que NO están en B: `new Set([...a].filter(x => !b.has(x)))`.
- Complemento (A'): Es la diferencia entre el Universo y el conjunto: `diferencia(universo, a)`.
- Conjunto Potencia $P(A)$: Calcula todos los subconjuntos usando operaciones a nivel de bits. Para un conjunto de tamaño N, hay 2^N subconjuntos posibles. Iteramos del 0 al $(2^N - 1)$ para armar cada subconjunto.

3. EXPLICACIÓN DE LA INTERFAZ Y REACTIVIDAD

La interfaz en 'index.vue' utiliza Vue 3 con la sintaxis moderna `<script setup lang="ts">`.

- Reactividad (ref): Usamos ref para capturar el texto que el usuario escribe. Cuando cambias los inputs, Vue actualiza todo automáticamente.
- Propiedades Computadas (computed): Los conjuntos U, A, B y los resultados son computed. Esto significa que solo se recalculan cuando el usuario cambia las entradas, ahorrando memoria y procesamiento.
- El Diagrama de Venn (SVG): Diseñamos círculos con posiciones fijas. Lo dinámico son sus estilos CSS. Una propiedad computada llamada 'vennColores' calcula qué color e intensidad de opacidad debe tener el círculo izquierdo, derecho y la intersección según el botón que presione el usuario.

BANCO DE PREGUNTAS CLAVE PARA LA EXPOSICIÓN

P1: ¿Por qué usaron un Set en lugar de un Array común en TypeScript?

R: Porque matemáticamente un conjunto no admite elementos repetidos, y la estructura Set del lenguaje filtra duplicados automáticamente. Además, buscar si un elemento existe con `.has()` en un Set es instantáneo $O(1)$, mientras que en un Array requiere recorrerlo $O(N)$.

P2: ¿Cómo funciona el algoritmo del Conjunto Potencia en operaciones de bits?

R: Usamos una máscara binaria. Si el conjunto tiene 3 elementos, habrá $2^3 = 8$ subconjuntos. Contamos del 0 al 7. La representación binaria de cada número nos dice si un elemento entra o no al subconjunto. Por ejemplo: 5 en binario es 101, por lo que el subconjunto tendrá al primer y tercer elemento.

P3: ¿Qué diferencia hay entre ref y computed en Vue 3?

R: ref define un valor reactivo básico que podemos modificar directamente. computed define un valor derivado (calculado) que cambia automáticamente cuando sus dependencias cambian. Las propiedades computadas tienen caché, por lo que no se ejecutan innecesariamente.

P4: ¿Cómo comprobaron que su motor matemático funciona sin fallas?

R: Implementamos 14 pruebas unitarias con Vitest que validan todas las operaciones. Comprobamos leyes matemáticas como la conmutatividad y la distributividad, asegurando que ante cualquier conjunto de entrada, el motor responda correctamente.